ECE 350: Real-Time Operating Systems Winter 2027

Lecture 3 — The Execution Model

Jeff Zarnett *2026-08-28*

So You Want to Run a C Program

To deeply understand how a process executes within the operating system, we have to start with making a program. It's an essential part of a process, after all, and how we build one matters for how it will run when the time comes. Then, a program needs to be loaded (and the process created) to start executing. While it's running, the process has various resources (as previously discussed), and we'll look at the stack frames and how functions are called, and a little bit about asynchronous control flow.

Compilation & Linking. Our experience with writing C programs is that they are compiled: there is a tool, the compiler, that turns the code that appears in the source files into something the computer can understand. That output is the executable code and some associated data, like we discussed when defining a process.

Many of the programs we write in a university context are small enough that the source all fits one file and maybe a header file. A real-world, large software project involves a lot more code than is going to fit in a single source file. If that's the case, we need to connect the different compiled output files to build the program. It's permissible for function `A()` in one file to call function `B()` in another file, where those two files get compiled into separate units. When compiling the unit that contains `A()`, the compiler must leave a note that says "when we figure out where `B()` is, put its address here" so that the call will actually work when the program runs.

We also spent extensive time in ECE 252 doing things that used libraries such as the `pthread` library, the standard C library. The library code does not get copy-pasted into the source of the executable; instead the binary you wrote references the library as it exists in the system. Here, the compiler, again, leaves a slightly different kind of note saying "at load time, figure out what the address of `write()` is, and put its address here" so that system call wrapper will work as intended.

Side note: you *can* copy-paste the library code into the executable. That's called *static linking* and while it's something that it is possible to do, it is not recommended in many situations [Dre]. With that said, languages like Go do this. Static linking makes the binary file bigger than it needs to be, sure, but it also means it is easy for a security vulnerability or performance bug to persist in your program. To fix such a vulnerability, you have to recompile, re-link, and then ship the changed binary. The alternative is *dynamic linking*, and that looks a lot more like "use the version of this library that you find on the system already". That's not perfect either – you can break many programs at once with a problem in the library – but we usually prefer dynamic linking either way.

The part of going back and filling in those pieces, or building the connections between the different parts of the program, is the post-compilation step called *linking*. Linking brings the different parts of the program together to create a cohesive program by resolving all those "we'll figure this out shortly" notes that it has left for its future self. There's compile-time linking for the first kind of note, where the location of `B()` is figured out at compile time and assigned. There is also dynamic linking, which is for the second kind of note, figuring out where `write()` is, and that happens later, when launching the process.

You experience a compile-time linking error any time you mistype a function name and the compiler says it has an undefined reference to function `do_tihng()` when clearly you meant it to be `do_thing()`. The compile-time linker does need to be assured of the existence of the library functions that appear in the second kind of note, otherwise it will say it does not know what function is being referenced. Only if the linker is able to resolve all notes of both kinds is there a valid executable produced.

Loading. Given a valid executable that we want to run (by request of the user or as the result of a `fork-exec` flow), we need to take the program and turn it into a process.

The concept of the process and process control block was mentioned in ECE 252 as part of understanding the backdrop, and revisited in the previous topic. In both places, the content focused on recognizing the difference between a program and a process and on what information about a process the operating system must know (and storing that in a data structure). This skips over the details about how to actually get the executable program into a state where it can run. That's what we cover here.

As we know, `fork()` makes an exact copy of a running process (with a new process control block and the like). We also covered the idea of `exec()` and its variants as the system call necessary to make a child process do something other than run the same code as its parent process. The `exec()` system call steps walk us through all the things necessary to turn a program that is not running into a running process.

The first thing that happens is throwing away all the things that are to be replaced: the executable program, memory, and other resources (with exceptions) all go away. Some things survive, like the process ID and even open file descriptors. After that, it is a blank canvas. So the OS can then load the compiled executable and start reading its contents to figure out how to set it up. Indeed, the compiled executable comes with a little setup guide, akin to IKEA instructions, for how to build its initial state. The OS should read them. Even if it is really good at building Kallax units after doing so like ten times.

References

[Dre] Ulrich Drepper. Static linking considered harmful. `https://www.akkadia.org/drepper/no_static_linking.html`. Online; accessed 27-August-2026.